

ANNOUNCEMENT

Introducing Pinecone Inference to streamline your AI workflow [Learn more >](#)

[← Learn](#)

Llama 2: AI Developers Handbook

 James Briggs

Jul 24, 2023

Share:

Deep Dives



Jump to section

[Getting Started](#)

[Building a Llama 2 Conversational Agent](#)

[References](#)

Llama 2 is the latest Large Language Model (LLM) from Meta AI. It is in many respects a groundbreaking release. First, Llama 2 is *open access* — meaning it is not closed behind an API and it's licensing allows *almost anyone* to use it and fine-tune new models on top of it. Second, Llama 2 is breaking records, scoring new benchmarks against all other "open access" models [1].

On this page you can find the latest best practices to using Llama 2. This is a living page and it will be regularly updated as we discover more.

Start using Pinecone for free

Pinecone is the developer-favorite vector database that's fast and easy to use at any scale.

[Subscribe](#)

Getting Started

Don't miss the next one... ×

Get an email the next time we publish an article about machine learning and similarity search.

[Get Updates](#)

We will begin at the beginning. How can we use Llama 2? The most flexible approach we can find is using Hugging Face Transformers. In the following examples we will be loading the largest of the Llama 2 models that has been fine-tuned for chat — the Llama-2-70b-chat-hf model. Nonetheless, the same methodology can be applied to use any of the Llama 2 models.

Access

Before we can do anything we must request access to Llama 2 models. We begin by filling an access form on Meta's website. You can expect your application to be approved fairly quickly (most report within an hour).

After some time you should receive an email stating you have been approved. Once you have this, head over to the Hugging Face model card for whichever model you'd like to use. You can find the model cards by navigating to "https://huggingface.co/<model_id>". A few of the model IDs currently accessible are:

- meta-llama/Llama-2-7b
- meta-llama/Llama-2-7b-chat-hf
- meta-llama/Llama-2-13b
- meta-llama/Llama-2-13b-chat-hf
- meta-llama/Llama-2-70b
- meta-llama/Llama-2-70b-chat-hf

The top of the model card should show another license to be accepted. It access form *must* be the same as that used on your Hugging Face account.

Don't miss the next one...

Get an email the next time we publish an article about machine learning and similarity search.

 Model card Files **Gated model** You have been granted access to this model

Access granted for gated model on Hugging Face

After some more waiting you should find a message on the model card stating *"You have been granted access to this model"*. This message means we have access, but there is one last step.

To download the models from Hugging Face we must authenticate ourselves. We can do this by heading to our Hugging Face *Settings > Access Tokens > New token* and creating a new **Read** token. We must copy this access token to place into our code later.

Once we've completed these steps, we're ready to jump into the code.

Building a Llama 2 Conversational Agent

Llama 2 is a rarity in open access models in that we can use the model as a *conversational agent* almost out of the box. A significant level of LLM performance is required to do this and this ability is usually reserved for closed-access LLMs like OpenAI's GPT-4.

Before we get started we should talk about system requirements. Llama 70B is a *big* model. We will load the model in the most optimal way currently possible but it still requires *at least* 35GB of GPU memory. That rules out almost everything except an A100 GPU which includes 40GB in the base model.

If you, like most people, are not able to source an A100 with a snap of your fingers — you can replicate the process with the 13B parameter version of Llama 2 (with just 15GB of GPU memory). Results will be less impressive but still good.

With all of that out of the way, let's begin.

Initializing the Model

All of the code we will be working through can be found [here](#).

We begin with ``pip install`` for all of the libraries we need to use.

```
1 !pip install -qU \  
2     transformers==4.31.0 \  
3     accelerate==0.21.0 \  
4     einops==0.6.1 \  
5     langchain==0.0.240 \  
6     xformers==0.0.20 \  
7     bitsandbytes==0.41.0
```



Don't miss the next one...

Get an email the next time we publish an article about machine learning and similarity search.

Before initializing the model itself we need to create two config objects. The first of those is our quantization config object `bnb_config`. This object enables us to quantize our model, allowing us to fit it onto a single A100 GPU — without quantization we would actually need ~7 A100s.

```
1 import torch
2 import transformers
3
4 # set quantization configuration to load large model with less GPU memory
5 # this requires the `bitsandbytes` library
6 bnb_config = transformers.BitsAndBytesConfig(
7     load_in_4bit=True,
8     bnb_4bit_quant_type='nf4',
9     bnb_4bit_use_double_quant=True,
10    bnb_4bit_compute_dtype=torch.bfloat16
11 )
```



The next configuration object uses the model configuration from Hugging Face itself to set different model parameters. To download this we first set our Hugging Face authentication key that we created earlier, then initialize the `model_config` using the `model_id`.

```
1 model_id = 'meta-llama/Llama-2-70b-chat-hf'
2
3 # begin initializing HF items, need auth token for these
4 hf_auth = '<YOUR_API_KEY>'
5 model_config = transformers.AutoConfig.from_pretrained(
6     model_id,
7     use_auth_token=hf_auth
8 )
```



We then download and initialize the model:

```
1 # initialize the model
2 model = transformers.AutoModelForCausalLM.from_pretrained(
3     model_id,
4     trust_remote_code=True,
5     config=model_config,
6     quantization_config=bnb_config,
7     device_map='auto',
8     use_auth_token=hf_auth
9 )
10 model.eval()
```



With that we're almost there but we're missing one step — the translation tokens. We perform this translation using a *tokenizer*. To initialize the Llan

Don't miss the next one...

Get an email the next time we publish an article about machine learning and similarity search.

```
1 tokenizer = transformers.AutoTokenizer.from_pretrained(  
2     model_id,  
3     use_auth_token=hf_auth  
4 )
```



Now we're ready to initialize the Hugging Face pipeline which will perform the full process from tokenization to text generation.

```
1 generate_text = transformers.pipeline(  
2     model=model, tokenizer=tokenizer,  
3     return_full_text=True, # langchain expects the full text  
4     task='text-generation',  
5     # we pass model parameters here too  
6     temperature=0.0, # 'randomness' of outputs, 0.0 is the min and 1.0 the max  
7     max_new_tokens=512, # max number of tokens to generate in the output  
8     repetition_penalty=1.1 # without this output begins repeating  
9 )
```



With that complete we're ready to begin implementing the model as a conversational agent using LangChain.

Conversational Agent in LangChain

With our Hugging Face pipeline initialized we can load in into LangChain like so:

```
1 from langchain.llms import HuggingFacePipeline  
2  
3 llm = HuggingFacePipeline(pipeline=generate_text)
```



Using `llm` we can now use Llama 2 with LangChain's advanced tooling such as agents, chains, and callbacks. We'll need to initialize a conversational agent which requires a few things, such as conversational `memory`, access to `tools`, and an `llm` (which we have already initialized).

We create our `memory` and `tools` like so:

```
1 from langchain.memory import ConversationBufferWindowMemory  
2 from langchain.agents import load_tools  
3  
4 memory = ConversationBufferWindowMemory(  
5     memory_key="chat_history", k=5, return_messages=True, output_key="c  
6 )  
7 tools = load_tools(["llm-math"], llm=llm)
```



Don't miss the next one...

Get an email the next time we publish an article about machine learning and similarity search.

With those ready, we initialize our conversational agent.



```
1 from langchain.agents import initialize_agent
2
3 # initialize agent
4 agent = initialize_agent(
5     agent="chat-conversational-react-description",
6     tools=tools,
7     llm=llm,
8     verbose=True,
9     early_stopping_method="generate",
10    memory=memory
11 )
```

Llama 2 Chat Prompt Structure

The Llama 2 chat model was fine-tuned for chat using a specific structure for prompts. This structure relied on four special tokens:

- `<s>`: the beginning of the entire sequence. `<<SYS>>\n` : the beginning of the system message.
- `\n</SYS>>\n\n` : the end of the system message.
- `[INST]` : the beginning of some instructions.
- `[/INST]`

With that in mind we would typically structure chat messages like so:



```
1 <s>[INST] <<SYS>>
2 You are a helpful, respectful and honest assistant. Always do...
3
4 If you are unsure about an answer, truthfully say "I don't know"
5 <</SYS>>
6
7 How many Llamas are we from skynet? [/INST]
```

During testing we actually found a slightly different structure to work. Which looks more like:

```
1 <<SYS>>
2 You are a helpful, respectful and honest assistant. Always do...
3
4 If you are unsure about an answer, truthfully say "I don't know"
5 <</SYS>>
6
7
```

Don't miss the next one...

Get an email the next time we publish an article about machine learning and similarity search.

[INST] Remember you are an assistant [/INST] User: How many Llamas are we from skynet?

Although this worked for us, we would suggest first trying the recommended structure from the Llama 2 paper.

To get the agent working we need it to output JSON format responses reliably. For this to work we encourage the use of JSON in the prompt and give several examples of how to do it — something we call *few-shot prompting*.

```
# special tokens used by llama 2 chat
B_INST, E_INST = "[INST]", "[/INST]"
B_SYS, E_SYS = "<<SYS>>\n", "\n<</SYS>>\n\n"

# create the system message
sys_msg = "<s>" + B_SYS + """Assistant is a expert JSON builder designed to assist with a wide range of tasks.

Assistant is able to respond to the User and use tools using JSON strings that contain "action" and "action_input" parameter

All of Assistant's communication is performed using this JSON format.

Assistant can also use tools by responding to the user with tool use instructions in the same "action" and "action_input" JSON object.

- "Calculator": Useful for when you need to answer questions about math.
- To use the calculator tool, Assistant should write like so:
    ```json
 {"action": "Calculator",
 "action_input": "sqrt(4)"}
    ```

Here are some previous conversations between the Assistant and User:

User: Hey how are you today?
Assistant: ```json
{"action": "Final Answer",
  "action_input": "I'm good thanks, how are you?"}
```

User: I'm great, what is the square root of 4?
Assistant: ```json
{"action": "Calculator",
 "action_input": "sqrt(4)"}
```

User: 2.0
Assistant: """ + E_SYS
```



Sign Up Free



```
User: Thanks could you tell me what 4 to the power of 2 is?
Assistant: ```json
{"action": "Calculator",
  "action_input": "4**2"}
```

User: 16.0
Assistant: ```json
{"action": "Final Answer",
 "action_input": "It looks like the answer is 16!"}
```
```

Here is the latest conversation between Assistant and User.""" + E_SYS

Don't miss the next one...

Get an email the next time we publish an article about machine learning and similarity search.

```

51 new_prompt = agent.agent.create_prompt(
52     system_message=sys_msg,
53     tools=tools
54 )
agent.agent.llm_chain.prompt = new_prompt

```

After setting our new system message we can move on to the prompt template for user messages.

In our tests we found the system message worked for encouraging the use of JSON responses but only for one or two interactions. Beyond this, Llama 2 chat seemed to forget about the JSON format.

This "forgetfulness" problem was mentioned in the Llama 2 paper.



In a dialogue setup, some instructions should apply for all the conversation turns, e.g., to respond succinctly, or to “act as” some public figure. When we provided such instructions to Llama 2-Chat, the subsequent response should always respect the constraint. However, our initial RLHF models tended to forget the initial instruction after a few turns of dialogue [1]

The authors found that they could improve this by adding instructions to each user message. We tried the same by added a brief reminder of the JSON format like so:

```

1 instruction = B_INST + " Respond to the following in JSON with 'action' and 'action_input' values " + E_INST
2 human_msg = instruction + "\nUser: {input}"
3
4 agent.agent.llm_chain.prompt.messages[2].prompt.template = human_msg

```



With that our agent should reliably produce agent-friendly JSON outputs. We can see this in the following queries:

In[26]:

```
1 agent("hey how are you today?")
```



```

[1m> Entering new AgentExecutor chain...[0m
[32;1m[1;3m

```

Don't miss the next one...

Get an email the next time we publish an article about machine learning and similarity search.


```
Assistant: ```json
{"action": "Final Answer",
 "action_input": "I'm good thanks, how are you?"}
```
```

```
[1m> Finished chain.
```

```
Out[26]:
```

```
{'input': 'hey how are you today?',
 'chat_history': [],
 'output': "I'm good thanks, how are you?"}
```

```
In[30]:
```

```
1 agent("what is 4 to the power of 2.1?")
```



```
Out[30]:
```

```
[1m> Entering new AgentExecutor chain...
```

```
[32;1m[1;3m
```

```
Assistant: ```json
```

```
{"action": "Calculator",
 "action_input": "4**2.1"}
```
```

```
Observation: [36;1m[1;3mAnswer: 18.37917367995256
```

```
Thought:[32;1m[1;3m
```

```
AI:
```

```
Assistant: ```json
```

```
{"action": "Final Answer",
 "action_input": "It looks like the answer is 18.37917367995256!"}
```
```

```
[1m> Finished chain.
```

```
Out[30]:
```

```
{'input': 'what is 4 to the power of 2.1?',
 'chat_history': [HumanMessage(content='hey how are you today?', additional_kwargs={}, example=False),
 AIMessage(content="I'm good thanks, how are you?", additional_kwargs={}, example=False)],
 'output': 'It looks like the answer is 18.37917367995256!'}
```

```
In[31]:
```

```
1 agent("can you multiply that by 3?")
```

Don't miss the next one...

Get an email the next time we publish an article about machine learning and similarity search.

Out[31]:

```

[1m> Entering new AgentExecutor chain...[0m
[32;1m[1;3m
Assistant: ```json
{"action": "Calculator",
 "action_input": "18.37917367995256 * 3"}
```[0m
Observation: [36;1m[1;3mAnswer: 55.13752103985769[0m
Thought:

```

Out[31]:

```

/usr/local/lib/python3.10/dist-packages/transformers/pipelines/base.py:1083: UserWarning: Yo
warnings.warn(

```

Out[31]:

```

[32;1m[1;3m

AI:
Assistant: ```json
{"action": "Final Answer",
 "action_input": "It looks like the answer is 55.13752103985769!"}
```[0m

[1m> Finished chain.[0m

```

Out[31]:

```

{'input': 'can you multiply that by 3?',
 'chat_history': [HumanMessage(content='hey how are you today?', additional_kwargs={}, examp
AIMessage(content='I'm good thanks, how are you?', additional_kwargs={}, example=False),
HumanMessage(content='what is 4 to the power of 2.1?', additional_kwargs={}, example=False)
AIMessage(content='It looks like the answer is 18.37917367995256!', additional_kwargs={},
'output': 'It looks like the answer is 55.13752103985769!'}

```

We get reliable results and thanks to quantization all of this is manageable within 38GB of GPU memory.

Llama 2 is the latest in a string of advances in a more *open* AI ecosystem. The models themselves are impressive. Yet, the real impact is likely to come from what the community does with them. In of innovation and new fine-tuned models thanks to the open access natu

### Don't miss the next one...

Get an email the next time we publish an article about machine learning and similarity search.

# References

[1] H. Touvron, L. Martin, K. Stone, et. al., [Llama 2: Open Foundation and Fine-Tuned Chat Models](#) (2023), Meta AI

Share:



## Further Reading

[LangGraph and Research Agents](#)

[Build Privacy-aware AI software using Pinecone](#)

[The Practitioner's Guide To E5](#)



### Product

- [Overview](#)
- [Documentation](#)
- [Integrations](#)
- [Trust and Security](#)

### Company

- [About](#)
- [Partners](#)
- [Careers](#)
- [Newsroom](#)
- [Contact](#)

### Solutions

- [Customers](#)
- [RAG](#)
- [Semantic Search](#)
- [Multi-Modal Search](#)
- [Candidate Generation](#)
- [Classification](#)

### Legal

- [Customer Terms](#)
- [Website Terms](#)
- [Privacy](#)
- [Cookies](#)
- [Cookie Preferences](#)

### Resources

- [Learning Center](#)
- [Community](#)
- [Pinecone Blog](#)
- [Support Center](#)
- [System Status](#)
- [What is a Vector Database?](#)
- [What is Retrieval Augmented Generation \(RAG\)?](#)

#### Don't miss the next one...

Get an email the next time we publish an article about machine learning and similarity search.

© Pinecone Systems, Inc. | San Francisco, CA

Pinecone is a registered trademark of Pinecone Systems, Inc.

**Don't miss the next one...**

Get an email the next time we publish an article about machine learning and similarity search.